

The logo for the National Laboratory of the Rockies, featuring the text "NATIONAL LABORATORY OF THE ROCKIES" in white, uppercase letters on a blue rectangular background.

NATIONAL LABORATORY OF THE ROCKIES

Tutorial: Parallel Sensitivity Analysis with SDOM

Overview

This tutorial explains how to run parallel sensitivity analysis using the SDOM optimization package. The script `example_session_2_parallel.py` demonstrates how to run multiple optimization simulations simultaneously, each with different storage CAPEX (capital expenditure) factors.

Why Parallel Processing?

When running sensitivity analysis, you typically need to run the same model multiple times with different parameter values. Running these simulations sequentially can be time-consuming. By using parallel processing, you can:

- **Reduce total execution time** by utilizing multiple CPU cores
- **Run independent simulations simultaneously** without waiting for each to complete
- **Efficiently explore parameter space** for sensitivity analysis

Required Packages

The script uses **built-in Python modules only**:

- `concurrent.futures` - For parallel processing (no installation needed!)
- `copy` - For copying data structures
- Standard libraries: `os`, `logging`, `subprocess`

External packages (already in your environment):

- `sdm` - The optimization package
- `highspy` - The solver

Script Structure

1. Imports and Setup

```
from concurrent.futures import ProcessPoolExecutor, as_completed
import copy

configure_logging(level=logging.INFO)
current_folder = os.getcwd()
```

What's happening:

- `ProcessPoolExecutor`: Creates separate Python processes for parallel execution
- `as_completed`: Allows us to process results as they finish (not necessarily in submission order)
- `configure_logging`: Sets up logging for the SDOM package

2. Define Sensitivity Analysis Parameters

```
storage_capex_factors = [1.0, 0.9, 0.8]
```

What's happening:

- We define a list of multiplication factors for the storage CAPEX
- `1.0` = baseline (100% of original cost)
- `0.9` = 10% reduction in storage cost
- `0.8` = 20% reduction in storage cost

You can add more factors to explore different scenarios, e.g., `[1.2, 1.0, 0.9, 0.8, 0.7, 0.5]`

3. The Simulation Function

```
def run_simulation_with_factor(factor, current_folder):
```

Purpose: This function runs a complete SDOM optimization with a specific storage CAPEX factor.

Key steps inside this function:

Step 3.1: Load the Data

```
data_dir = os.path.join(current_folder, "sample_data", "br_test_daily_b")  
data = load_data(data_dir)
```

Each parallel process loads its own copy of the data.

Step 3.2: Modify Storage CAPEX

```
if "storage_data" in data and "P_capex" in data["storage_data"].index:  
    data["storage_data"].loc["P_capex"] = data["storage_data"].loc["P_capex"] *  
    factor
```

What's happening:

- Locates the `P_capex` row in the `storage_data` DataFrame
- Multiplies **all columns** of that row by the factor
- For example, if original `P_capex` = [100, 200, 150] and factor = 0.8, new values = [80, 160, 120]

Step 3.3: Initialize and Run Model

```
model = initialize_model(
    data,
    n_hours=n_steps,
    with_resilience_constraints=with_resilience_constraints,
    model_name=f'SDOM_training_factor_{factor}'
)

best_result = run_solver(model, solver_dict)
```

What's happening:

- Creates a Pyomo optimization model with the modified data
- Runs the HiGHS solver to find the optimal solution
- Each simulation gets a unique model name for identification

Step 3.4: Export Results

```
if best_result:
    export_results(model, case, output_dir=output_dir+"\\")
    return (factor, True, output_dir)
else:
    return (factor, False, output_dir)
```

What's happening:

- If optimization succeeds, exports results to a unique directory
- Returns a tuple with: (factor used, success status, output directory)
- Each factor gets its own output folder, e.g., `sample_results_br_test_daily_b-factor0.8`

4. The Main Parallel Execution

```
def main():
    max_workers = min(len(storage_capex_factors), os.cpu_count() or 1)

    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        future_to_factor = {
            executor.submit(run_simulation_with_factor, factor, current_folder):
            factor
            for factor in storage_capex_factors
        }
```

```
for future in as_completed(future_to_factor):
    factor = future_to_factor[future]
    try:
        result = future.result()
        results.append(result)
    except Exception as exc:
        print(f"Simulation with factor {factor} generated an exception:
{exc}")
```

What's happening step by step:

Step 4.1: Determine Number of Workers

```
max_workers = min(len(storage_capex_factors), os.cpu_count()-1 or 1)
```

- Checks how many CPU cores are available
- Uses the smaller of: number of factors or number of CPU cores - 1
- Example: If you have 8 cores and 3 factors, it uses 3 workers

Step 4.2: Submit All Tasks

```
future_to_factor = {
    executor.submit(run_simulation_with_factor, factor, current_folder): factor
    for factor in storage_capex_factors
}
```

- Creates a dictionary mapping Future objects to their factors
- Each `executor.submit()` starts a new process running the simulation function
- All tasks are submitted immediately and run in parallel

Step 4.3: Collect Results as They Complete

```
for future in as_completed(future_to_factor):
    result = future.result()
    results.append(result)
```

- `as_completed()` yields futures as they finish (not in submission order)
- The fastest simulation will be processed first
- Error handling ensures one failed simulation doesn't stop others

5. Results Summary

```

successful = [r for r in results if r[1]]
failed = [r for r in results if not r[1]]

print(f"\nTotal simulations: {len(results)}")
print(f"Successful: {len(successful)}")
print(f"Failed: {len(failed)}")

```

What's happening:

- Separates successful and failed simulations
- Prints a comprehensive summary with output directories
- Helps you quickly identify which scenarios completed successfully

How to Use This Script

Step 1: Modify Parameters (Optional)

Edit the factors you want to test:

```
storage_capex_factors = [1.2, 1.0, 0.8, 0.6, 0.4]
```

Step 2: Run the Script

```
python example_session_2_parallel.py
```

Step 3: Monitor Progress

You'll see output like:

```

=====
Starting simulation with storage_capex_factor = 1.0
=====

Original P_capex values:
...
Modified P_capex values (factor=1.0):
...

```

Step 4: Check Results

Results are saved in separate directories:

- `sample_results_br_test_daily_b-factor1.0/`
- `sample_results_br_test_daily_b-factor0.9/`

- `sample_results_br_test_daily_b-factor0.8/`

Each contains:

- `OutputGeneration_*.csv`
- `OutputStorage_*.csv`
- `OutputSummary_*.csv`
- `OutputThermalGeneration_*.csv`

Understanding Parallel Execution Flow

Sequential (Original Script)

```
Factor 1.0 → [=====] → Results
                ↓
Factor 0.9 → [=====] → Results
                ↓
Factor 0.8 → [=====] → Results

Total Time: 3 × simulation_time
```

Parallel (New Script)

```
Factor 1.0 → [=====] → Results
Factor 0.9 → [=====] → Results
Factor 0.8 → [=====] → Results

Total Time: ≈ simulation_time (if you have 3+ cores)
```

Advanced Customization

Change Number of Parallel Workers

Manually set the number of workers:

```
max_workers = 4 # Use 4 cores regardless of factors
```

Add More Parameters to Vary

Modify the function to accept additional parameters:

```
def run_simulation_with_factor(factor, n_steps, current_folder):
    # Now you can vary both factor and n_steps
```

Process Results After Completion

Add analysis after all simulations complete:

```
# In main() after collecting results
for factor, success, output_dir in successful:
    if success:
        # Load and compare results
        gen_data = pd.read_csv(f"{output_dir}/OutputGeneration_*.csv")
        # Perform comparison or plotting
```

Common Issues and Solutions

Issue 1: Import Errors in Child Processes

Problem: Child processes can't import `sdm` module

Solution: Ensure the script is run from the correct directory with:

```
import sys
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
```

Issue 2: Memory Usage

Problem: Running too many simulations simultaneously uses too much RAM

Solution: Limit the number of workers:

```
max_workers = 2 # Only run 2 simulations at a time
```

Issue 3: Logging Conflicts

Problem: Multiple processes writing to the same log file

Solution: Configure separate log files per process or use console logging only (already done in this script)

Performance Tips

1. **Optimal Worker Count:** Usually set to number of physical CPU cores (not threads)
2. **Memory Monitoring:** Each process needs its own memory for data and model
3. **I/O Bottleneck:** If loading data is slow, consider pre-loading and passing data as arguments
4. **Result Storage:** Ensure output directory has sufficient disk space

Comparing with Sequential Execution

To compare performance, you can time both approaches:

Parallel:

```
python example_session_2_parallel.py  
# Check total execution time
```

Sequential:

```
python example_session_2.py # Run 3 times manually changing the factor  
# Sum up the three execution times
```

Expected speedup: $\sim 2\text{-}3\times$ with 3+ CPU cores (depending on I/O overhead)

Next Steps

After running the sensitivity analysis, you can:

1. **Compare Results:** Load all output files and create comparison plots
2. **Analyze Trends:** See how storage CAPEX affects total system cost, generation mix, etc.
3. **Extend Analysis:** Add more parameters (e.g., wind CAPEX, solar capacity factors)
4. **Automate Post-Processing:** Create a script to automatically compare and visualize results
5. **You can adapt this template** for various sensitivity analyses in SDOM or other optimization models!